

INFORME TÉCNICO

Desarrollo de dos programas de consola en Java
con estructuras condicionales (Actividad 01 y Actividad 02)

1. INTRODUCCIÓN

El presente informe describe el desarrollo de dos programas de consola elaborados en Java que resuelven problemas de decisión mediante estructuras condicionales. La Actividad 01 simula el sistema de ventas de una tienda que comercializa cuadernos por docena. La Actividad 02 modela la venta de pasajes de una empresa de transporte que cubre la ruta Lima – Huancayo.

Cada actividad constituye un proyecto independiente de NetBeans que emplea Ant como sistema de construcción, por lo que incluye su archivo build.xml, su manifest.mf y la carpeta nbproject. Dentro de la carpeta src se crea un paquete propio: Tienda para la primera actividad y Transporte para la segunda.

En ambos casos el código se separa en dos clases con responsabilidades bien definidas. La clase modelo almacena los datos y ejecuta los cálculos, mientras que la clase controladora contiene el método main y se ocupa de interactuar con el usuario a través del teclado y la pantalla. Esta separación permite modificar la forma de solicitar los datos sin alterar las reglas de negocio.

Además de explicar el diseño, el informe incorpora el código de los métodos más relevantes, las tablas de reglas, los casos de prueba y las salidas reales obtenidas al ejecutar ambos programas, de manera que cada resultado pueda verificarse mediante una operación manual.

2. OBJETIVOS

2.1 Objetivo general

Desarrollar dos aplicaciones de consola en Java, organizadas en clases modelo y controladora, que calculen importes, descuentos y obsequios aplicando estructuras condicionales, y documentar su diseño, funcionamiento y verificación.

2.2 Objetivos específicos

1. Construir la clase Producto y la clase AplicaProducto para calcular la compra de cuadernos por docena con su descuento y sus lapiceros de obsequio.
2. Construir la clase Pasaje y la clase AplicaPasaje para calcular la compra de pasajes de Lima a Huancayo con su descuento y sus caramelos de obsequio.
3. Dividir cada cálculo en métodos pequeños, de modo que cada método resuelva un solo paso y devuelva el valor calculado.
4. Diferenciar correctamente la condición de la tienda, que entrega obsequio desde S/. 200, de la del transporte, que solo lo entrega por encima de S/. 200.
5. Validar los datos de entrada con ciclos do-while para impedir precios, cantidades u opciones inválidas.
6. Permitir atender a varios clientes en una misma ejecución mediante un contador y una pregunta de continuidad.
7. Presentar en consola un recibo o boleta con explicaciones condicionales claras para el cliente.
8. Comprobar el funcionamiento con casos de prueba que incluyan los valores límite de cada regla.

3. ENUNCIADOS Y REGLAS

3.1 Actividad 01: tienda de cuadernos

Una tienda vende cuadernos por docena y ofrece un descuento sobre el importe de la compra según la cantidad de docenas adquiridas. Además, obsequia lapiceros según el importe que el cliente debe pagar después de aplicar el descuento. El programa recibe el precio de la docena y la cantidad de docenas, y determina el importe de la compra, el importe del descuento, el importe a pagar y la cantidad de lapiceros de obsequio.

Condición	Regla aplicada
10 docenas o más	Descuento del 20% sobre el importe de la compra
Menos de 10 docenas	Descuento del 10% sobre el importe de la compra
Importe a pagar mayor o igual que S/. 200	2 lapiceros por cada docena comprada
Importe a pagar menor que S/. 200	Ningún lapicero

3.2 Actividad 02: pasajes de Lima a Huancayo

Una empresa de transporte cubre la ruta de Lima a Huancayo en dos turnos, mañana y noche, y en ambos el pasaje cuesta S/. 37.50. La empresa aplica un descuento según la cantidad de pasajes y obsequia caramelos según el importe que se debe pagar. El programa recibe el turno y la cantidad de pasajes, y determina el importe de la compra, el descuento, el importe a pagar y los caramelos de obsequio.

Condición	Regla aplicada
15 pasajes o más	Descuento del 8%
Menos de 15 pasajes	Descuento del 5%
Importe a pagar mayor que S/. 200	2 caramelos por cada boleto
Importe a pagar menor o igual que S/. 200	Ningún caramelo

Diferencia importante. En la tienda un pago de exactamente S/. 200 sí da derecho a los lapiceros, porque la condición usa mayor o igual. En el transporte un pago de exactamente S/. 200 no da derecho a los caramelos, porque la condición usa solo mayor. En ambas actividades el obsequio se evalúa con el importe ya descontado, nunca con el importe bruto.

El documento de la Actividad 01 describe con detalle la estructura de clases que debía seguirse. Para la Actividad 02 se aplicó exactamente el mismo diseño, cambiando los datos y las reglas por los del enunciado del transporte.

4. HERRAMIENTAS Y ENTORNO

Herramienta	Uso en el trabajo
Java	Lenguaje de programación de ambas aplicaciones
Apache NetBeans	Entorno para crear, editar, compilar y ejecutar los proyectos
Ant	Sistema de construcción de cada proyecto, con build.xml y la carpeta nbproject
Scanner	Lectura de datos desde el teclado, configurado con el Locale de Estados Unidos
String.format	Formato de los importes con dos decimales, por ejemplo S/. 240.00
Consola de NetBeans	Ventana donde se ingresan los datos y se observan los resultados

5. ORGANIZACIÓN DE LOS PROYECTOS

Se crearon dos proyectos separados. Cada uno tiene la estructura estándar que genera NetBeans para una aplicación Java con Ant, y dentro de src se agrega el paquete correspondiente con sus dos clases.

```

Actividad01/
  build.xml
  manifest.mf
  nbproject/
  src/
    Tienda/
      Producto.java
      AplicaProducto.java

Actividad02/
  build.xml
  manifest.mf
  nbproject/
  src/
    Transporte/
      Pasaje.java
      AplicaPasaje.java

```

Elemento	Responsabilidad
build.xml	Define las tareas de compilación, limpieza y empaquetado del proyecto
manifest.mf	Indica datos del archivo JAR que se genera
nbproject	Guarda la configuración interna que NetBeans necesita para abrir el proyecto
Producto y Pasaje	Clases modelo con los atributos, los cálculos y el recibo
AplicaProducto y AplicaPasaje	Clases controladoras con el método main, las validaciones y el ciclo de clientes

Todas las clases emplean comentarios en español y bloques separados con encabezados como ATRIBUTOS, CONSTRUCTOR, MÉTODOS DE CÁLCULO y MÉTODO PRINCIPAL. Esto facilita localizar rápidamente cada parte del código.

6. DISEÑO DE LAS CLASES

6.1 Clases modelo

Las clases Producto y Pasaje representan una venta. Separan tres grupos de atributos: las constantes, los datos de entrada y los datos de resultado. Los datos de entrada llegan por el constructor y pueden modificarse con setters, mientras que los resultados se calculan dentro de la clase y solo tienen getters, de modo que ninguna otra clase pueda alterarlos por error.

Grupo	Producto (Actividad 01)	Pasaje (Actividad 02)
Constante	NOMBRE_PRODUCTO = "Cuaderno"	PRECIO_PASAJE = 37.50
Entrada	precioDocena, cantidadDocenas	turno, cantidadPasajes
Resultado	importeCompra, porcentajeDescuento, importeDescuento, importeAPagar	importeCompra, porcentajeDescuento, importeDescuento, importeAPagar
Obsequio	cantidadLapiceros	cantidadCaramelos

6.2 Clases controladoras

AplicaProducto y AplicaPasaje no contienen reglas de descuento. Su trabajo es mostrar la bienvenida, pedir los datos, crear el objeto del modelo, pedirle que procese la compra y mostrar el resultado. También controlan el ciclo que permite atender a más de un cliente.

6.3 Métodos de cálculo

Cada paso de la operación tiene su propio método, que calcula un valor, lo guarda en su atributo y lo devuelve. El método procesarCompra los llama en el orden que exige el enunciado, porque cada paso depende del anterior.

Nº	Método	Operación realizada
1	calcularImporteCompra()	Precio unitario por cantidad
2	calcularPorcentajeDescuento()	Elige el porcentaje según la cantidad
3	calcularImporteDescuento()	Importe de la compra por el porcentaje

N°	Método	Operación realizada
4	calcularImporteAPagar()	Importe de la compra menos el descuento
5	calcularCantidadLapiceros() o calcularCantidadCaramelos()	Aplica la regla del obsequio sobre el importe a pagar
6	procesarCompra()	Ejecuta los cinco pasos en orden

7. DESARROLLO DE LA ACTIVIDAD 01

7.1 Clase Producto

El nombre del producto es una constante privada y estática porque la tienda siempre vende el mismo artículo. El constructor recibe solamente el precio de la docena y la cantidad de docenas; los cinco resultados quedan en cero hasta que se invoque procesarCompra.

```
private static final String NOMBRE_PRODUCTO = "Cuaderno";

private double precioDocena;
private int cantidadDocenas;

private double importeCompra;
private double porcentajeDescuento;
private double importeDescuento;
private double importeAPagar;
private int cantidadLapiceros;

public Producto(double precioDocena, int cantidadDocenas) {
    this.precioDocena = precioDocena;
    this.cantidadDocenas = cantidadDocenas;
}
```

7.2 Cálculo del descuento y del obsequio

El porcentaje se elige con un if-else sobre la cantidad de docenas. El obsequio se decide con otro if-else, pero sobre el importe a pagar, que ya incluye el descuento. La condición usa mayor o igual, por lo que S/. 200 exactos sí otorgan lapiceros.

```
public double calcularPorcentajeDescuento() {
    if (cantidadDocenas >= 10) {
        porcentajeDescuento = 0.20;
    } else {
        porcentajeDescuento = 0.10;
    }
    return porcentajeDescuento;
}

public double calcularImporteAPagar() {
    importeAPagar = importeCompra - importeDescuento;
    return importeAPagar;
}

public int calcularCantidadLapiceros() {
    if (importeAPagar >= 200) {
        cantidadLapiceros = 2 * cantidadDocenas;
    } else {
        cantidadLapiceros = 0;
    }
    return cantidadLapiceros;
}
```

7.3 Método procesarCompra

```

public void procesarCompra() {
    calcularImporteCompra();
    calcularPorcentajeDescuento();
    calcularImporteDescuento();
    calcularImporteAPagar();
    calcularCantidadLapiceros();
}

```

Si se cambiara el orden, el programa fallaría sin avisar: por ejemplo, calcular el descuento antes que el importe de la compra multiplicaría por cero. Por eso el orden importe, porcentaje, descuento, pago y lapiceros queda fijado dentro de este único método.

7.4 Recibo de compra

El método mostrarResultado imprime un recibo decorado con líneas de signos igual. Muestra el nombre del producto, el precio por docena y las docenas compradas con el formato `String.format(Locale.US, "S/. %.2f", valor)`. Después presenta dos explicaciones condicionales: una sobre el descuento, que felicita al cliente si obtuvo el 20% o le sugiere comprar diez docenas para subir a ese porcentaje, y otra sobre el obsequio, que indica cuántos lapiceros recibe o explica que no llegó a S/. 200. El recibo se cierra con el mensaje de agradecimiento enmarcado.

7.5 Clase AplicaProducto

La clase controladora declara un Scanner estático con `useLocale(Locale.US)`, lo que permite escribir precios con punto decimal sin importar la configuración regional del equipo. El método mostrarBienvenida explica las condiciones vigentes antes de pedir cualquier dato. En el main, un ciclo do-while externo atiende a los clientes uno tras otro, y un contador `numeroCliente` se muestra en cada vuelta. Dentro de cada vuelta, dos ciclos do-while internos repiten la pregunta mientras el valor ingresado sea menor o igual que cero.

```

do {
    numeroCliente++;
    System.out.println("\nATENDIENDO AL CLIENTE N° " + numeroCliente);

    double precio;
    do {
        System.out.print("Ingrese el precio de la docena (S/.): ");
        precio = teclado.nextDouble();
    } while (precio <= 0);

    int docenas;
    do {
        System.out.print("Ingrese la cantidad de docenas: ");
        docenas = teclado.nextInt();
    } while (docenas <= 0);

    Producto producto = new Producto(precio, docenas);
    producto.procesarCompra();
    producto.mostrarResultado();

    System.out.print("¿Desea atender a otro cliente? (S = Sí / N = No): ");
    respuesta = Character.toUpperCase(teclado.next().charAt(0));
    teclado.nextLine();
} while (respuesta == 'S');

```

La respuesta se lee con `teclado.next()`, se convierte a mayúscula y se toma solo el primer carácter, así que s, S o Sí continúan el ciclo. La instrucción `teclado.nextLine()` posterior limpia el salto de línea que queda pendiente en el buffer. Al terminar, el programa informa cuántos clientes fueron atendidos.

7.6 Ejecución

Recibo obtenido al ingresar un precio de S/. 25 y 12 docenas:

```

=====
                RECIBO DE COMPRA
=====
Producto          : Cuaderno
Precio por docena  : S/. 25.00
Docenas compradas : 12
=====
Importe de la compra : S/. 300.00
¡Felicidades! Por comprar 10 o más docenas obtiene un 20% de descuento.
Descuento aplicado (20%) : S/. 60.00
>>> IMPORTE A PAGAR : S/. 240.00 <<<
¡Felicidades! Por pagar S/. 200 o más recibe 2 lapiceros por docena: 24 lapiceros de obsequio.
=====
                ¡Gracias por su compra!
=====

```

8. DESARROLLO DE LA ACTIVIDAD 02

8.1 Clase Pasaje

La estructura es la misma de Producto. La diferencia principal es que el precio del pasaje es una constante de la clase, PRECIO_PASAJE = 37.50, y no un dato que el usuario ingrese. Como ambos turnos cuestan lo mismo, el turno se guarda solo para mostrarlo en la boleta y no influye en el cálculo.

```

private static final double PRECIO_PASAJE = 37.50;

private String turno;
private int cantidadPasajes;

public Pasaje(String turno, int cantidadPasajes) {
    this.turno = turno;
    this.cantidadPasajes = cantidadPasajes;
}

```

8.2 Cálculo del descuento y del obsequio

El descuento es del 8% desde 15 pasajes y del 5% por debajo de esa cantidad. Para los caramelos la comparación es estricta, de modo que un pago de exactamente S/. 200 no entrega ninguno.

```

public double calcularPorcentajeDescuento() {
    if (cantidadPasajes >= 15) {
        porcentajeDescuento = 0.08;
    } else {
        porcentajeDescuento = 0.05;
    }
    return porcentajeDescuento;
}

public int calcularCantidadCaramelos() {
    if (importeAPagar > 200) {
        cantidadCaramelos = 2 * cantidadPasajes;
    } else {
        cantidadCaramelos = 0;
    }
    return cantidadCaramelos;
}

```

8.3 Clase AplicaPasaje

El controlador tiene un paso adicional respecto a la tienda: pedir el turno. Se muestra un menú con las opciones 1 para Mañana y 2 para Noche, y un do-while repite la pregunta hasta que se escriba una opción válida. Después se pide la cantidad de pasajes con la misma validación de la actividad anterior.

```

int opcion;
do {
    System.out.print("Elija el turno (1 = Mañana / 2 = Noche): ");
    opcion = teclado.nextInt();
} while (opcion != 1 && opcion != 2);
String turno = (opcion == 1) ? "Mañana" : "Noche";

```

8.4 Ejecución

Boleta obtenida al elegir el turno Mañana y 16 pasajes:

```

=====
          BOLETA DE VIAJE
=====
Ruta           : Lima a Huancayo
Turno          : Mañana
Precio por pasaje : S/. 37.50
Pasajes comprados : 16
=====
Importe de la compra : S/. 600.00
¡Felicidades! Por comprar 15 o más pasajes obtiene un 8% de descuento.
Descuento aplicado (8%) : S/. 48.00
>>> IMPORTE A PAGAR : S/. 552.00 <<<
¡Felicidades! Por pagar más de S/. 200 recibe 2 caramelos por boleto: 32 caramelos de obsequio.
=====
          ¡Buen viaje y gracias!
=====

```

9. COMPARACIÓN ENTRE AMBAS ACTIVIDADES

Aspecto	Actividad 01: Tienda	Actividad 02: Transporte
Paquete y clases	Tienda: Producto y AplicaProducto	Transporte: Pasaje y AplicaPasaje
Precio	Lo ingresa el usuario	Constante de S/. 37.50
Cantidad	Docenas	Pasajes
Umbral del descuento	10 docenas	15 pasajes
Porcentajes	20% y 10%	8% y 5%
Regla del obsequio	Pago mayor o igual que S/. 200	Pago mayor que S/. 200
Cantidad del obsequio	2 lapiceros por docena	2 caramelos por boleto
Dato adicional	Ninguno	Turno con menú de dos opciones

10. PRUEBAS REALIZADAS

Los dos programas se compilaron y se ejecutaron con varios clientes seguidos, ingresando los datos por teclado. En cada caso se comparó el resultado de la consola con el cálculo manual. Se eligieron valores cercanos a los límites de las reglas, porque allí es donde más fácilmente se comete un error.

10.1 Actividad 01

Caso	Compra	Descuento	Importe desc.	A pagar	Lapiceros
S/. 25 × 12 docenas	300.00	20%	60.00	240.00	24
S/. 20 × 5 docenas	100.00	10%	10.00	90.00	0
S/. 25 × 9 docenas	225.00	10%	22.50	202.50	18
S/. 20 × 10 docenas	200.00	20%	40.00	160.00	0
S/. 30 × 2 docenas	60.00	10%	6.00	54.00	0

El caso de 9 docenas comprueba que, aunque no llega al 20%, el pago de S/. 202.50 supera el umbral y entrega 18 lapiceros. El caso de 10 docenas comprueba lo contrario: la compra bruta es de S/. 200, pero después del descuento se paga S/. 160 y no hay obsequio. Esto confirma que la regla se evalúa sobre el importe final.

10.2 Actividad 02

Caso	Compra	Descuento	Importe desc.	A pagar	Caramelos
Mañana, 16 pasajes	600.00	8%	48.00	552.00	32
Noche, 5 pasajes	187.50	5%	9.375	178.125	0
Mañana, 6 pasajes	225.00	5%	11.25	213.75	12
Noche, 14 pasajes	525.00	5%	26.25	498.75	28
Mañana, 15 pasajes	562.50	8%	45.00	517.50	30

Los casos de 14 y 15 pasajes muestran el cambio de 5% a 8% en el límite exacto. El caso de 5 pasajes queda por debajo de S/. 200 y no entrega caramelos, mientras que el de 6 pasajes lo supera y entrega 12.

10.3 Validación de datos

Dato ingresado	Programa	Comportamiento observado
Precio de la docena: -3	Actividad 01	Muestra el aviso y vuelve a pedir el precio
Precio de la docena: 0	Actividad 01	Muestra el aviso y vuelve a pedir el precio
Cantidad de docenas: -1	Actividad 01	Muestra el aviso y vuelve a pedir la cantidad
Cantidad de docenas: 0	Actividad 01	Muestra el aviso y vuelve a pedir la cantidad
Turno: 3	Actividad 02	Indica que la opción no es válida y repite el menú
Cantidad de pasajes: 0	Actividad 02	Muestra el aviso y vuelve a pedir la cantidad
Respuesta final: N	Ambas	Termina y muestra el total de clientes atendidos

11. OBSERVACIONES TÉCNICAS

Redondeo en pantalla. Los importes se guardan como double y se muestran con dos decimales. En la prueba de 5 pasajes el descuento exacto es 9.375 y el pago exacto es 178.125, pero la consola muestra 9.38 y 178.13. El valor almacenado no cambia; solo el formato redondea al imprimir.

Uso de double. El enunciado pide atributos de tipo double, por lo que se respetó esa decisión. Para un sistema comercial real convendría usar BigDecimal, que evita los pequeños errores de representación de los números decimales binarios.

Entradas no numéricas. Los ciclos do-while rechazan valores menores o iguales que cero, pero si el usuario escribe letras en lugar de un número, Scanner lanza una excepción InputMismatchException. Una mejora posible sería capturarla con try-catch y volver a pedir el dato.

Limpieza del buffer. Después de leer con next se ejecuta nextLine para consumir el salto de línea. Sin esa instrucción, una lectura de texto posterior recibiría una línea vacía.

Responsabilidades separadas. Como los cálculos están en la clase modelo, se podrían reutilizar sin cambios en una interfaz gráfica o en una aplicación web.

12. PASOS PARA EJECUTAR LOS PROYECTOS EN NETBEANS

1. Abrir Apache NetBeans y elegir la opción para abrir un proyecto, seleccionando la carpeta Actividad01 o Actividad02.
2. Verificar en la ventana de proyectos que el paquete Tienda o Transporte aparezca dentro de Source Packages con sus dos clases.
3. Hacer clic derecho sobre el proyecto y elegir Clean and Build para compilarlo con Ant.
4. Abrir la clase controladora, AplicaProducto o AplicaPasaje, hacer clic derecho y elegir Run File.
5. Escribir los datos en la ventana de salida de NetBeans y pulsar Enter después de cada uno.
6. Responder S para atender a otro cliente o N para finalizar el programa.

13. CONCLUSIONES

1. Las estructuras if-else permiten expresar con claridad los descuentos por tramos y las condiciones de obsequio de ambos enunciados.

2. Dividir el cálculo en métodos de un solo paso, ordenados por procesarCompra, hace que el programa sea fácil de leer, de probar y de corregir.
3. La diferencia entre mayor o igual y mayor a S/. 200 es un detalle pequeño en el código, pero cambia el resultado en el caso límite, por lo que debe respetarse exactamente como lo indica cada enunciado.
4. Evaluar el obsequio con el importe ya descontado evita entregar obsequios que el cliente no merece, como se comprobó con 10 docenas de S/. 20.
5. Los ciclos do-while de validación protegen al programa de precios, cantidades u opciones inválidas, y el ciclo externo permite atender a varios clientes en una misma ejecución.
6. Separar el modelo del controlador ordena el proyecto y deja las reglas de negocio listas para reutilizarse en otros tipos de aplicación.
7. Las pruebas con valores límite dieron los resultados esperados en las dos actividades, y coinciden con el cálculo manual.